

Los prefijos de JSTL en JSP

Núcleo `c`, formato `fmt`, base de datos `sql`, XML `x`, funciones `fn` — y el prefijo estándar `jsp`:

Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación — Aplicaciones Web

Jakarta Standard Tag Library 3.0 ■ Junio de 2026

Fundamentos

- ▶ ¿Qué es JSTL y por qué existe?
- ▶ El adiós a los *scriptlets*
- ▶ La directiva `taglib` y los espacios de nombres
- ▶ Las cinco bibliotecas + el prefijo `jsp:`
- ▶ Dependencias Maven

Un bloque por prefijo

- ▶ `jsp:` acciones estándar de JSP
- ▶ `c` núcleo (*core*)
- ▶ `fmt` formato e internacionalización
- ▶ `sql` acceso a base de datos
- ▶ `x` procesamiento de XML
- ▶ `fn` funciones para EL
- ▶ ¿Y `h:?` — aclaración importante

¿Qué es JSTL?

Definición

La **Jakarta Standard Tag Library** (JSTL) es un conjunto estandarizado de etiquetas que cubren las tareas más comunes de una página JSP: condicionales, iteración, formato, internacionalización, acceso a datos y manejo de XML.

- ▶ Sustituye el código Java incrustado (*scriptlets*) por etiquetas declarativas y legibles.
- ▶ Trabaja de la mano del **lenguaje de expresiones** (EL): `${...}`.
- ▶ Separa responsabilidades: la JSP *presenta*; la lógica vive en servlets y clases Java.
- ▶ Versión actual: **JSTL 3.0** (Jakarta EE), con el espacio de nombres `jakarta.*`.

El problema: *scriptlets* frente a JSTL

Antes — Java incrustado en la vista (difícil de leer y mantener):

```
<%-- scriptlet: mezcla logica y presentacion --%>
<% for (Estudiante e : (List<Estudiante>) request.getAttribute("lista")) { %>
    <tr><td><%= e.getNombre() %></td></tr>
<% } %>
```

Después — JSTL + EL (declarativo, sin Java a la vista):

```
<c:forEach var="e" items="${lista}">
    <tr><td><c:out value="${e.nombre}" /></td></tr>
</c:forEach>
```

- Menos errores, código más corto y vistas que cualquiera puede leer.

La directiva taglib y los espacios de nombres

Cada biblioteca se habilita con una directiva al inicio de la JSP:

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
```

Migración javax → jakarta

En Jakarta EE los URI cambiaron de `http://java.sun.com/jsp/jstl/...` a URN cortos `jakarta.tags...`. Si copia ejemplos antiguos con `java.sun.com`, las etiquetas **no se procesarán** en Tomcat 10 o superior.

- ▶ El **prefijo** (`c`, `fmt`, ...) es una convención: usted podría renombrarlo, pero no debe hacerlo.
- ▶ El **URI** es fijo e identifica la biblioteca real.

Las cinco bibliotecas JSTL + el prefijo jsp:

Prefijo	URI (Jakarta, JSTL 3.0)	Para qué sirve
c	<code>jakarta.tags.core</code>	Núcleo: variables, condición, iteración, URL
fmt	<code>jakarta.tags.fmt</code>	Formato de números/fechas e i18n
sql	<code>jakarta.tags.sql</code>	Consultas a base de datos (solo prototipo)
x	<code>jakarta.tags.xml</code>	Procesar XML con XPath
fn	<code>jakarta.tags.functions</code>	Funciones (sobre todo de texto) para EL
jsp	<i>(integrado, sin taglib)</i>	Acciones estándar del propio JSP

- ▶ **jsp**: no es JSTL: viene con la especificación de JSP.
- ▶ **h**: no aparece aquí: pertenece a Jakarta Faces (JSF), no a JSTL.

Dependencias Maven (Jakarta JSTL 3.0)

El servidor (Tomcat 11) aporta Servlet/JSP; JSTL se agrega como dependencia: API + implementación.

```
<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
  <version>3.0.2</version>
</dependency>
<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>
  <version>3.0.1</version>
</dependency>
```

- ▶ Olvidar la **implementación** (segunda dependencia) causa el clásico *Unable to find taglib*.

jsp: — Acciones estándar de JSP (no es JSTL)

Importante

El prefijo `jsp:` viene **integrado** en la especificación de JSP: no requiere directiva `taglib`. Permite acciones predefinidas sin escribir Java.

Acción	Función
<code>jsp:useBean</code>	Crea o recupera un JavaBean en un ambito
<code>jsp:setProperty</code>	Asigna propiedades del bean (desde parametros)
<code>jsp:getProperty</code>	Lee una propiedad del bean
<code>jsp:include</code>	Incluye otro recurso en tiempo de ejecucion
<code>jsp:forward</code>	Reenvia la peticion a otro recurso
<code>jsp:param</code>	Pasa parametros a include/forward
<code>jsp:text</code> , <code>jsp:element</code>	Generan texto o elementos dinamicos

► Hoy `useBean/getProperty` se sustituyen casi siempre por EL (`${bean.prop}`).

jsp: — useBean, setProperty, getProperty

```
<%-- Crea un bean "estudiante" en el ambito de la peticion --%>
<jsp:useBean id="estudiante"
             class="ec.edu.uteq.appweb.modelo.Estudiante"
             scope="request"/>

<%-- Asigna todas las propiedades cuyo nombre coincide con un parametro --%>
<jsp:setProperty name="estudiante" property="*/>

<%-- Lee una propiedad (equivale a EL ${estudiante.nombre}) --%>
Nombre: <jsp:getProperty name="estudiante" property="nombre"/>
```

jsp: — include, forward, param

```
<%-- Incluye una cabecera en tiempo de ejecucion --%>
<jsp:include page="cabecera.jsp">
  <jsp:param name="titulo" value="Estudiantes"/>
</jsp:include>

<%-- Reenvia el control a otro recurso (no vuelve aqui) --%>
<jsp:forward page="lista.jsp"/>
```

- `jsp:include` (acción) se resuelve en *ejecución*; la directiva `<%@ include %>` lo hace en *compilación*.

c — Núcleo (*core*): la biblioteca esencial

`jakarta.tags.core` — la más usada. Cuatro grupos de etiquetas:

Variables y salida

- ▶ `c:out` — imprime y **escapa** (anti-XSS)
- ▶ `c:set` — crea o asigna variables
- ▶ `c:remove` — elimina una variable
- ▶ `c:catch` — captura excepciones

Condicionales

- ▶ `c:if`
- ▶ `c:choose` / `c:when` /
`c:otherwise`

Iteración

- ▶ `c:forEach` — recorre colecciones
- ▶ `c:forTokens` — recorre cadenas divididas

URL

- ▶ `c:url` — construye URL con la ruta de contexto
- ▶ `c:param` — agrega parámetros
- ▶ `c:import` — importa un recurso
- ▶ `c:redirect` — redirige

c — out, set, remove, catch

```
<%-- c:out escapa la salida: defensa anti-XSS --%>
<c:out value="${estudiante.nombre}" default="(sin nombre)"/>

<%-- c:set crea variables en un ambito (page, request, session, application) --%>
<c:set var="iva" value="${0.15}" scope="request"/>

<%-- c:remove elimina una variable --%>
<c:remove var="iva"/>

<%-- c:catch atrapa una excepcion sin romper la pagina --%>
<c:catch var="error">
    <c:set var="x" value="${10 / 0}"/>
</c:catch>
<c:if test="${not empty error}">Ocurrio un error.</c:if>
```

c — Condicionales: c:if y c:choose

```
<%-- c:if: condicion simple (no tiene "else") --%>
<c:if test="${empty lista}">
  <p>No hay estudiantes registrados.</p>
</c:if>

<%-- c:choose: varias ramas, como switch/if-else --%>
<c:choose>
  <c:when test="${estudiante.carrera == 'Software'}">
    <span>Carrera principal</span>
  </c:when>
  <c:when test="${empty estudiante.carrera}">
    <span>Sin carrera asignada</span>
  </c:when>
  <c:otherwise>
    <span>Otra carrera</span>
  </c:otherwise>
</c:choose>
```

c — Iteración: c:forEach y c:forEachTokens

```
<!-- Recorre una coleccion; varStatus expone indice, count, first, last --%>
<table>
<c:forEach var="e" items="${lista}" varStatus="st">
  <tr class="${st.index % 2 == 0 ? 'par' : 'impar'}">
    <td>${st.count}</td>
    <td><c:out value="${e.nombre}"/></td>
  </tr>
</c:forEach>
</table>

<!-- Itera un rango numerico --%>
<c:forEach var="i" begin="1" end="5">${i} </c:forEach>

<!-- Divide una cadena por separadores y recorre los trozos --%>
<c:forEachTokens var="t" items="rojo,verde,azul" delims=",">
  ${t}
</c:forEachTokens>
```

c — URL: c:url, c:param, c:redirect

```
<%-- c:url antepone la ruta de contexto y codifica parametros --%>
<c:url var="enlaceEditar" value="/estudiantes">
  <c:param name="accion" value="editar"/>
  <c:param name="id" value="${e.id}"/>
</c:url>
<a href="${enlaceEditar}">Editar</a>

<%-- c:redirect envia al navegador a otra direccion --%>
<c:redirect url="/estudiantes"/>
```

- c:url evita errores cuando cambia el nombre o la ubicación de la aplicación.

`jakarta.tags.fmt` — presenta números y fechas según la configuración regional, y traduce textos.

Números y fechas

- ▶ `fmt:formatNumber`
- ▶ `fmt:parseNumber`
- ▶ `fmt:formatDate`
- ▶ `fmt:parseDate`

Localización

- ▶ `fmt:setLocale`
- ▶ `fmt:bundle` / `fmt:setBundle`
- ▶ `fmt:message` / `fmt:param`
- ▶ `fmt:timeZone` / `fmt:setTimeZone`
- ▶ `fmt:requestEncoding`

```
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>

<%-- Moneda y porcentaje --%>
<fmt:formatNumber value="${1234.5}" type="currency"/>
<fmt:formatNumber value="${0.15}" type="percent"/>

<%-- Numero con dos decimales --%>
<fmt:formatNumber value="${promedio}" minFractionDigits="2"
maxFractionDigits="2"/>

<%-- Fecha y hora con patron --%>
<fmt:formatDate value="${estudiante.creado}"
pattern="dd/MM/yyyy HH:mm"/>
```

```
<%-- Selecciona idioma y archivo de propiedades (mensajes_es.properties) --%>
<fmt:setLocale value="es_EC"/>
<fmt:setBundle basename="mensajes"/>

<%-- Muestra el texto segun el idioma activo --%>
<h1><fmt:message key="titulo.estudiantes"/></h1>

<%-- Mensaje con parametros: "Bienvenido, {0}" --%>
<fmt:message key="saludo">
  <fmt:param value="${estudiante.nombre}"/>
</fmt:message>
```

- Los textos viven en archivos .properties; cambiar de idioma no toca la JSP.

`jakarta.tags.sql` — ejecuta consultas SQL directamente en la página.

Etiquetas

`sql:setDataSource`, `sql:query`, `sql:update`, `sql:param`, `sql:dateParam`,
`sql:transaction`.

Advertencia pedagógica — úsela solo para aprender

Colocar SQL en la vista **rompe la separación por capas**. En proyectos reales el acceso a datos va en un DAO (capa de servidor), no en la JSP. La biblioteca `sql` es útil para una demostración rápida o un prototipo, **nunca para producción**.

sql — query, update, param (solo demostración)

```
<%@ taglib prefix="sql" uri="jakarta.tags.sql" %>

<sql:setDataSource var="ds" driver="org.postgresql.Driver"
    url="jdbc:postgresql://localhost:5432/appweb2026ppa"
    user="postgres" password="P@$7Gr3$QL"/>

<!-- Consulta con parametro enlazado (anti-inyeccion) --%>
<sql:query var="rs" dataSource="${ds}">
    SELECT id, nombre, correo FROM estudiantes WHERE carrera = ?
    <sql:param value="${param.carrera}"/>
</sql:query>

<c:forEach var="fila" items="${rs.rows}">
    ${fila.nombre} - ${fila.correo}<br/>
</c:forEach>
```

`jakarta.tags.xml` — analiza y recorre documentos XML usando expresiones **XPath** (atributo `select`).

Lectura

- ▶ `x:parse` — analiza el XML
- ▶ `x:out` — imprime un nodo
- ▶ `x:set` — guarda un nodo en variable

Lógica y transformación

- ▶ `x:if`, `x:choose` / `x:when` / `x:otherwise`
- ▶ `x:forEach`
- ▶ `x:transform` / `x:param` (XSLT)

x — parse, forEach, out con XPath

```
<%@ taglib prefix="x" uri="jakarta.tags.xml" %>

<c:import var="xmlDoc" url="/datos/estudiantes.xml"/>
<x:parse var="doc" doc="{xmlDoc}"/>

<%-- Recorre nodos con XPath y los imprime --%>
<ul>
<x:forEach select="$doc/estudiantes/estudiante" var="e">
  <li><x:out select="$e/nombre"/> -
    <x:out select="$e/carrera"/></li>
</x:forEach>
</ul>

<%-- Condicion con XPath --%>
<x:if select="count($doc/estudiantes/estudiante) > 0">
  Hay estudiantes en el XML.
</x:if>
```

`jakarta.tags.functions` — no son etiquetas, sino **funciones** que se invocan dentro de `${...}`, sobre todo para texto.

Función	Devuelve
<code>fn:length(x)</code>	longitud de cadena o colección
<code>fn:contains(s,t)</code>	si s contiene t
<code>fn:startsWith / endsWith</code>	si empieza / termina con un texto
<code>fn:indexOf(s,t)</code>	posición de t en s
<code>fn:substring(s,a,b)</code>	subcadena entre dos posiciones
<code>fn:substringBefore / After</code>	parte previa / posterior a un texto
<code>fn:replace(s,a,b)</code>	reemplaza a por b
<code>fn:split(s,d) / fn:join(a,d)</code>	divide / une por un separador
<code>fn:toUpperCase / toLowerCase</code>	cambia mayúsculas/minúsculas
<code>fn:trim(s)</code>	quita espacios de los extremos
<code>fn:escapeXml(s)</code>	escapa caracteres XML/HTML (anti-XSS)

```
<%@ taglib prefix="fn" uri="jakarta.tags.functions" %>

<%-- Longitud y condicion --%>
<c:if test="${fn:length(lista) == 0}">Lista vacia.</c:if>

<%-- Texto a mayusculas --%>
${fn:toUpperCase(estudiante.nombre)}

<%-- Buscar dentro de un texto (sin distinguir mayusculas) --%>
<c:if test="${fn:containsIgnoreCase(estudiante.correo, 'uteq')}">
    Correo institucional
</c:if>

<%-- Escapar salida manualmente (equivale a c:out) --%>
${fn:escapeXml(estudiante.nombre)}
```

¿Y el prefijo h:? — Aclaración importante

h: NO pertenece a JSTL

El prefijo h: es de **Jakarta Faces (JSF)**, otra tecnología de vista distinta de JSP+JSTL. Su URI es `jakarta.faces.html` e incluye etiquetas como `h:form`, `h:inputText` o `h:commandButton`.

Tecnología	Prefijos típicos	Naturaleza
JSP + JSTL	c, fmt, sql, x, fn, jsp	Etiquetas sobre páginas JSP
Jakarta Faces (JSF)	h, f, ui	Componentes de interfaz (Facelets)

- Se mencionan juntos por confusión frecuente; en una JSP con JSTL **no se usan etiquetas h:.**

Cuándo usar cada biblioteca — mapa de decisión

Necesito...	Uso...
mostrar datos, condicionar o iterar	<code>c</code> (núcleo)
formatear números/fechas o traducir	<code>fmt</code>
recorrer o transformar un XML	<code>x</code>
manipular cadenas dentro de EL	<code>fn</code>
un prototipo rápido contra la BD	<code>sql</code> (nunca en producción)
crear un bean o incluir/reenviar	<code>jsp:</code> (estándar)

Regla de oro

La vista **presenta**; nunca contiene reglas de negocio ni acceso a datos. `c` y `fn` cubren el 90 % de las páginas; `fmt` aporta el formato; `x` y `sql` son casos especiales.

- ▶ JSTL reemplaza los *scriptlets* por etiquetas declarativas que trabajan con EL.
- ▶ Cinco bibliotecas: `c`, `fmt`, `sql`, `x`, `fn`, cada una con su URI `jakarta.tags.*`.
- ▶ `jsp:` es del propio JSP (acciones estándar); `h:` es de JSF, no de JSTL.
- ▶ Siempre escape la salida (`c:out` o `fn:escapeXml`) y mantenga el SQL fuera de la vista.

¿Preguntas?

- ▶ Eclipse Foundation. *Jakarta Standard Tag Library 3.0*.
<https://jakarta.ee/specifications/tags/3.0/>
- ▶ Eclipse Foundation. *Jakarta Server Pages 4.0*.
<https://jakarta.ee/specifications/pages/4.0/>
- ▶ Apache Software Foundation. *Apache Tomcat 11.0 Documentation*.
<https://tomcat.apache.org/tomcat-11.0-doc/>